

Comm-Log Send Reconciliation

Data Analyst Take-Home — Merchant 501 | October 2026

1. Executive Summary

Objective. Reproduce Finance's target_base of 22 qualifying sends for merchant 501 during October 2026 using the supplied SQLite database and the reporting rules in the README.

Final result: 22. The initial raw count was 30. The difference comes from one unapproved campaign branch and duplicate attempts inside two retry chains.

2. Scope and Reporting Rules

- Merchant: 501.
- Period: October 2026 (sent_time >= 2026-10-01 and sent_time < 2026-11-01).
- Communication type: '2' (Campaign).
- A campaign is reportable only when its creation workflow is finalized and processing_status = 'processed'.
- A retry is represented by campaign.parent_id pointing to the previous campaign. All campaigns in one retry chain represent the same underlying communication.
- Within a retry chain, a customer is counted once even if several send attempts were made.
- A standalone campaign is different: every send row is an independent event, so a legitimate repeat of the same customer is retained.

3. Data Used

File / Table	Role	How it was used
Data Analyst Take Home Assignment.pdf	Assignment brief	Defines target_base = 22 and required deliverables.
README.md	Data dictionary / business rules	Defines campaign eligibility, retry semantics and standalone behavior.
campaign	Campaign master data	Provides campaign IDs, parent_id and approval/processing status.
communication_log	Send-level data	Provides customer-level send attempts and timestamps.
comm_log.db	SQLite database	Database used to run the investigation and final SQL.
generate_dataset.py	Dataset generator	Supporting context for the constructed retry and standalone cases.

4. Investigation

I started with the simplest count that matched the assignment scope. The result was 30, which did not match Finance's 22, so I checked campaign eligibility and then traced retry relationships.

4.1 Campaign eligibility check

Campaign	Parent	Creation status	Processing	Interpretation
9001	—	approved	processed	Eligible; root of Family A
9002	9001	approved	processed	Eligible retry
9003	9002	approved	processed	Eligible retry
9004	9001	approval_awaiting	processed	Not reportable
9101	—	approved	processed	Eligible standalone
9201	—	approved	processed	Eligible; root of Family B
9202	9201	approved	processed	Eligible retry

First adjustment: Campaign 9004 is still approval_awaiting. It has 4 communication_log rows, but those rows are not included in official reporting. This changes 30 to 26.

4.2 Retry Family A — 9001 → 9002 → 9003

The parent_id chain shows that 9002 is a retry of 9001 and 9003 is a retry of 9002. The send log contains 13 attempts across the three campaign rows, but only 10 distinct customers.

Customer	9001	9002	9003
C1	Delivered	—	—
C2	Failed	Delivered	—
C3	Failed	Failed	Delivered
C4–C10	Delivered	—	—

Second adjustment: 13 attempts collapse to 10 distinct customers because C2 and C3 were retried within the same underlying communication. Running total: 26 → 23.

4.3 Retry Family B — 9201 → 9202

Campaign 9202 is a retry of 9201. There are 6 attempts across the family but only 5 distinct customers. D1 failed on 9201 and was successfully retried on 9202.

Customer	9201	9202
D1	Failed	Delivered
D2–D5	Delivered	—

Third adjustment: 6 attempts become 5 distinct customers within this retry chain, reducing 23 → 22.

4.4 Standalone Campaign 9101

Campaign 9101 has no parent and no retry child. It contributes 7 send events. Customer C20 appears twice, on October 10 and October 20, and both are successful. Because 9101 is standalone, this is a legitimate repeat event rather than a retry duplicate, so both events remain in the count.

5. Reconciliation Bridge

Step	Adjustment / Description	Result	Reason
0	Naive qualifying-send count	30	Starting point from communication_log.
1	Exclude campaign 9004	26	9004 is approval_awaiting and therefore not reportable; 4 rows are removed.
2	Deduplicate retry Family A (9001→9002→9003)	23	13 attempts represent 10 distinct customers; remove 3 retry duplicates.
3	Deduplicate retry Family B (9201→9202)	22	6 attempts represent 5 distinct customers; remove 1 retry duplicate.
Final	Target base	22	Matches Finance's stated target_base.

6. Final Composition of target_base

Underlying communication	Counting method	Contribution
Family A: 9001 → 9002 → 9003	Distinct customers	10
Family B: 9201 → 9202	Distinct customers	5
Standalone: 9101	Every send event	7
Total		22

7. Final SQL

The final query uses the campaign table to determine eligibility and recursively resolves eligible retry chains to their root. Retry families are counted by distinct customer; standalone roots are counted by send event.

```
WITH RECURSIVE
eligible_campaigns AS (
SELECT id, parent_id
FROM campaign
WHERE merchant_id = 501
AND creation_status IN ('approved', 'aborted', 'resumed', 'stopped')
AND processing_status = 'processed'
),
campaign_tree(id, root_id) AS (
SELECT id, id
FROM eligible_campaigns
WHERE parent_id IS NULL

UNION ALL

SELECT c.id, t.root_id
FROM eligible_campaigns c
JOIN campaign_tree t
ON c.parent_id = t.id
),
scoped_logs AS (
SELECT l.customer_id, t.root_id
FROM communication_log l
JOIN campaign_tree t
ON t.id = l.communication_id
WHERE l.merchant_id = 501
AND l.sent_time >= '2026-10-01'
AND l.sent_time < '2026-11-01'
AND l.communication_type = '2'
),
roots_with_retries AS (
SELECT DISTINCT root_id
FROM campaign_tree
WHERE id <> root_id
),
family_counts AS (
SELECT
root_id,
CASE
WHEN root_id IN (SELECT root_id FROM roots_with_retries)
THEN COUNT(DISTINCT customer_id)
ELSE COUNT(*)
END AS qualifying_sends
FROM scoped_logs
GROUP BY root_id
)
SELECT COALESCE(SUM(qualifying_sends), 0) AS target_base
FROM family_counts;
```

Verified result against the supplied SQLite database: target_base = 22.

8. Data Observation

One detail that initially looked like a duplicate was customer C20 appearing twice under campaign 9101. The two sends occurred on October 10 and October 20, and both were successful. The README's distinction between a retry and an independent re-targeting event explains why both rows should remain. This shows why deduplication cannot be applied just by customer_id; the campaign's retry structure has to be considered first.

9. Conclusion

The Finance target_base of 22 can be reproduced from the raw data. Starting from 30 qualifying communication-log rows, 4 rows are removed because campaign 9004 has not cleared approval. The remaining retry families are then reconciled at the customer level: Family A contributes 10 distinct customers and Family B contributes 5. Standalone campaign 9101 contributes 7 individual send events. The final total is therefore $10 + 5 + 7 = 22$.

Appendix A — Investigation SQL

Query 1 — Naive count

```
SELECT COUNT(*)
FROM communication_log
WHERE merchant_id = 501
AND sent_time >= '2026-10-01'
AND sent_time < '2026-11-01'
AND communication_type = '2';
```

Result: 30

New DatabaseOpen DatabaseWrite ChangesRevert ChangesUndoOpen ProjectSave Project

Database StructureBrowse DataEdit PragmasExecute SQL

SQL 1*

1-- STEP 0: Count all qualifying communication-log rows

2SELECT COUNT(*)

3FROM communication_log

4WHERE merchant_id = 501

5AND sent_time >= '2026-10-01'

6AND sent_time < '2026-11-01'

7AND communication_type = '2';

8

COUNT(*)

130

Execution finished without errors.

Result: 1 rows returned in 11ms

At line 1:

-- STEP 0: Count all qualifying communication-log rows

SELECT COUNT(*)

Query 2 — Campaign status check

```
SELECT id, name, creation_status, processing_status
FROM campaign
WHERE merchant_id = 501
ORDER BY id;
```

Result: 7 campaigns; 9004 is approval_awaiting.

```
8
9
10  -- STEP 1: Inspect campaign IDs, names, and eligibility-related statuses
11  SELECT
12      id,
13      name,
14      creation_status,
15      processing_status
16  FROM campaign
17  WHERE merchant_id = 501
18  ORDER BY id;
19
```

	id	name	creation_status	processing_status
1	9001	Diwali Cart Recovery - Wave 1	approved	processed
2	9002	Diwali Cart Recovery - Retry A	approved	processed
3	9003	Diwali Cart Recovery - Retry B	approved	processed
4	9004	Diwali Cart Recovery - Retry C ...	approval_awaiting	processed
5	9101	Diwali Flash Sale - Standalone	approved	processed
6	9201	Diwali Wave 2	approved	processed
7	9202	Diwali Wave 2 - Retry	approved	processed

```
Execution finished without errors.
Result: 7 rows returned in 21ms
At line 10:
-- STEP 1: Inspect campaign IDs, names, and eligibility-related statuses
SELECT
```

Query 3 — Campaign 9004 sends

```
SELECT COUNT(*)
FROM communication_log
WHERE merchant_id = 501
AND communication_id = 9004
AND sent_time >= '2026-10-01'
AND sent_time < '2026-11-01'
AND communication_type = '2';
```

Result: 4

Query 4 — Exclude 9004

```
SELECT COUNT(*)
FROM communication_log
WHERE merchant_id = 501
AND sent_time >= '2026-10-01'
AND sent_time < '2026-11-01'
AND communication_type = '2'
AND communication_id != 9004;
```

Result: 26

```

19
20
21 -- STEP 1: Recalculate the count after excluding ineligible campaign 9004
22 SELECT COUNT(*)
23 FROM communication_log
24 WHERE merchant_id = 501
25        AND sent_time >= '2026-10-01'
26        AND sent_time < '2026-11-01'
27        AND communication_type = '2'
28        AND communication_id != 9004;

```

	COUNT(*)
1	26

Execution finished without errors.
Result: 1 rows returned in 8ms
At line 21:
-- STEP 1: Recalculate the count after excluding ineligible campaign 9004
SELECT COUNT(*)

Query 5 — Retry relationships

```

SELECT id, parent_id, name, creation_status, processing_status
FROM campaign
WHERE merchant_id = 501
ORDER BY id;

```

Result: 9001→9002→9003, 9201→9202; 9101 standalone.

```

29
30
31 -- STEP 2: Inspect parent_id to identify retry chains
32 SELECT
33     id,
34     parent_id,
35     name,
36     creation_status,
37     processing_status
38 FROM campaign
39 WHERE merchant_id = 501
40 ORDER BY id;

```

	id	parent_id	name	creation_status	processing_status
1	9001	NULL	Diwali Cart Recovery - Wave 1	approved	processed
2	9002	9001	Diwali Cart Recovery - Retry A	approved	processed
3	9003	9002	Diwali Cart Recovery - Retry B	approved	processed
4	9004	9001	Diwali Cart Recovery - Retry C ...	approval_awaiting	processed
5	9101	NULL	Diwali Flash Sale - Standalone	approved	processed
6	9201	NULL	Diwali Wave 2	approved	processed
7	9202	9201	Diwali Wave 2 - Retry	approved	processed

Execution finished without errors.
Result: 7 rows returned in 15ms
At line 31:
-- STEP 2: Inspect parent_id to identify retry chains
SELECT

Query 6 — Repeated customers in Family A

```

SELECT customer_id, COUNT(*) AS attempts
FROM communication_log
WHERE merchant_id = 501

```

Result: C2 = 2 attempts; C3 = 3 attempts.

```
41
42
43 -- STEP 2: Find customers appearing multiple times in retry family 9001 → 9002 → 9003
44 SELECT
45     customer_id,
46     COUNT(*) AS attempts
47 FROM communication_log
48 WHERE merchant_id = 501
49     AND communication_id IN (9001, 9002, 9003)
50     AND sent_time >= '2026-10-01'
51     AND sent_time < '2026-11-01'
52     AND communication_type = '2'
53 GROUP BY customer_id
54 HAVING COUNT(*) > 1
55 ORDER BY customer_id;
```

	customer_id	attempts
1	C2	2
2	C3	3

Execution finished without errors.
Result: 2 rows returned in 12ms
At line 43:
-- STEP 2: Find customers appearing multiple times in retry family 9001 → 9002 → 9003
SELECT

Query 7 — Family A distinct customers

```
SELECT COUNT(DISTINCT customer_id) AS family_a_customers
FROM communication_log
WHERE merchant_id = 501
    AND communication_id IN (9001, 9002, 9003)
    AND sent_time >= '2026-10-01'
    AND sent_time < '2026-11-01'
    AND communication_type = '2';
```

Result: 10

```

58
59
60 -- STEP 2: Count distinct customers across retry family 9001 → 9002 → 9003
61 SELECT COUNT(DISTINCT customer_id) AS family_a_customers
62 FROM communication_log
63 WHERE merchant_id = 501
64     AND communication_id IN (9001, 9002, 9003)
65     AND sent_time >= '2026-10-01'
66     AND sent_time < '2026-11-01'
67     AND communication_type = '2';

```

	family_a_customers
1	10

```

Execution finished without errors.
Result: 1 rows returned in 8ms
At line 60:
-- STEP 2: Count distinct customers across retry family 9001 → 9002 → 9003
SELECT COUNT(DISTINCT customer_id) AS family_a_customers

```

Query 8 — Family B distinct customers

```

SELECT COUNT(DISTINCT customer_id) AS family_b_customers
FROM communication_log

```

```

WHERE merchant_id = 501
AND communication_id IN (9201, 9202)
AND sent_time >= '2026-10-01'
AND sent_time < '2026-11-01'
AND communication_type = '2';

```

Result: 5

```

69
70 -- STEP 3: Count distinct customers across retry family 9201 → 9202
71 SELECT COUNT(DISTINCT customer_id) AS family_b_customers
72 FROM communication_log
73 WHERE merchant_id = 501
74     AND communication_id IN (9201, 9202)
75     AND sent_time >= '2026-10-01'
76     AND sent_time < '2026-11-01'
77     AND communication_type = '2';
78
79
80 -- STEP 4: Count every send event for standalone campaign 9101
81 SELECT COUNT(*) AS standalone_events
82 FROM communication_log
83 WHERE merchant_id = 501
84     AND communication_id = 9101
85

```

family_b_customers

1	5
---	---

Execution finished without errors.

Result: 1 rows returned in 12ms

At line 70:

```

-- STEP 3: Count distinct customers across retry family 9201 → 9202
SELECT COUNT(DISTINCT customer_id) AS family_b_customers
FROM communication_log
WHERE merchant_id = 501
    AND communication_id IN (9201, 9202)
    AND sent_time >= '2026-10-01'

```

Query 9 — Standalone events

```

SELECT COUNT(*) AS standalone_events
FROM communication_log
WHERE merchant_id = 501
AND communication_id = 9101
AND sent_time >= '2026-10-01'
AND sent_time < '2026-11-01'
AND communication_type = '2';

```

Result: 7

```

78
79
80 -- STEP 4: Count every send event for standalone campaign 9101
81 SELECT COUNT(*) AS standalone_events
82 FROM communication_log
83 WHERE merchant_id = 501
84        AND communication_id = 9101
85        AND sent_time >= '2026-10-01'
86        AND sent_time < '2026-11-01'
87        AND communication_type = '2';
88
89
90

```

	standalone_events
1	7

```

Execution finished without errors.
Result: 1 rows returned in 6ms
At line 80:
-- STEP 4: Count every send event for standalone campaign 9101
SELECT COUNT(*) AS standalone_events
FROM communication_log
WHERE merchant_id = 501
      AND communication_id = 9101
      AND sent_time >= '2026-10-01'

```

Query 10 — C20 investigation

```

SELECT id, communication_id, customer_id, delivery_status, sent_time, scheduled_time
FROM communication_log
WHERE merchant_id = 501
AND customer_id = 'C20'
ORDER BY sent_time;

```

Result: two successful sends on Oct 10 and Oct 20.

Query 11 — Family A attempts

```

SELECT communication_id, customer_id, delivery_status, sent_time
FROM communication_log
WHERE merchant_id = 501
AND communication_id IN (9001, 9002, 9003)
ORDER BY customer_id, sent_time;

```

Result: 13 rows.

Query 12 — Family B attempts

```

SELECT communication_id, customer_id, delivery_status, sent_time
FROM communication_log
WHERE merchant_id = 501
AND communication_id IN (9201, 9202)
ORDER BY customer_id, sent_time;

```

Result: 6 rows.

Query 13 — Final investigation check

```

WITH RECURSIVE
eligible_campaigns AS (
SELECT id, parent_id
FROM campaign
WHERE merchant_id = 501
AND creation_status IN ('approved', 'aborted', 'resumed', 'stopped')
AND processing_status = 'processed'
),
campaign_tree(id, root_id) AS (
SELECT id, id
FROM eligible_campaigns
WHERE parent_id IS NULL

```

```

),
family_counts AS (
SELECT
root_id,
CASE
WHEN root_id IN (SELECT root_id FROM roots_with_retries)
THEN COUNT(DISTINCT customer_id)
ELSE COUNT(*)
END AS qualifying_sends
FROM scoped_logs
GROUP BY root_id
)
SELECT COALESCE(SUM(qualifying_sends), 0) AS target_base
FROM family_counts;

```

Result: 22

```

129
130 -- FINAL INVESTIGATION CHECK:
131 -- Family A distinct customers + Family B distinct customers + Standalone events
132
133 SELECT
134 (
135     SELECT COUNT(DISTINCT customer_id)
136     FROM communication_log
137     WHERE merchant_id = 501
138         AND communication_id IN (9001, 9002, 9003)
139         AND sent_time >= '2026-10-01'
140         AND sent_time < '2026-11-01'
141         AND communication_type = '2'
142 )
143 +
144 (
145     SELECT COUNT(DISTINCT customer_id)

```

target_base

1 22

Execution finished without errors.

Result: 1 rows returned in 8ms

At line 130:

-- FINAL INVESTIGATION CHECK:

-- Family A distinct customers + Family B distinct customers + Standalone events

SELECT

```

(
    SELECT COUNT(DISTINCT customer_id)
    FROM communication_log
    WHERE merchant_id = 501

```

```

129
130 -- FINAL INVESTIGATION CHECK:
131 -- Family A distinct customers + Family B distinct customers + Standalone events
132
133 SELECT
134 (
135     SELECT COUNT(DISTINCT customer_id)
136     FROM communication_log
137     WHERE merchant_id = 501
138           AND communication_id IN (9001, 9002, 9003)
139           AND sent_time >= '2026-10-01'
140           AND sent_time < '2026-11-01'
141           AND communication_type = '2'
142 )
143 +
144 (
145     SELECT COUNT(DISTINCT customer_id)
146     FROM communication_log
147     WHERE merchant_id = 501
148           AND communication_id IN (9201, 9202)
149           AND sent_time >= '2026-10-01'
150           AND sent_time < '2026-11-01'
151           AND communication_type = '2'
152 )
153 +
154 (
155     SELECT COUNT(*)
156     FROM communication_log
157     WHERE merchant_id = 501
158           AND communication_id = 9101
159           AND sent_time >= '2026-10-01'
160           AND sent_time < '2026-11-01'
161           AND communication_type = '2'
162 ) AS target_base;

```

target_base

1 22

Execution finished without errors.

Result: 1 rows returned in 8ms

At line 130:

-- FINAL INVESTIGATION CHECK:

-- Family A distinct customers + Family B distinct customers + Standalone events